WYDZIAŁ ELEKTRONIKI I TECHNIK INFORMACYJNYCH

Sieci Wielousługowe (SWUS)

Projekt – Zarządzanie politykami sieciowymi w systemie Kubernetes

Zespół P1

Igor Skiba 331530

Radosław Rytel-Kuc 331524

25 stycznia 2026, Warszawa

Spis treści

1. Etap 1: Utworzenie klastra Kubernetes (Milestone 1) . . . 2
1.1. Środowisko testowe . . . 2
1.2. Wyłączenie mechanizmu SWAP . . . 2
1.3. Konfiguracja parametrów jądra systemu . . . 2
1.4. Środowisko uruchomieniowe kontenerów . . . 2
1.5. Instalacja narzędzi Kubernetes . . . 3
1.6. Inicjalizacja klastra na węźle control-plane . . . 3
1.7. Dołączenie węzłów roboczych . . . 3
1.8. Instalacja CNI – Cilium . . . 3
1.9. Weryfikacja działania klastra . . . 4
2. Etap 2: Wdrożenie aplikacji na klastrze Kubernetes . . . 5
2.1. Konfiguracja Deploymentów . . . 5
2.2. Weryfikacja wdrożenia . . . 6
2.3. Wstępne testy komunikacji (brak polityk) . . . 7
3. Etap 3. Stworzenie polityk sieciowych (Network Policies): . . . 8
3.1. Założenia i implementacja . . . 8
3.2. Wdrożenie polityk sieciowych . . . 9
3.3. Testowanie polityk sieciowych: . . . 9
3.3.1. Połączenia dozwolone . . . 9
3.3.2. Połączenia zablokowane . . . 10
3.4. Refleksja nad politykami bezpieczeństwa . . . 10
Literatura . . . 11

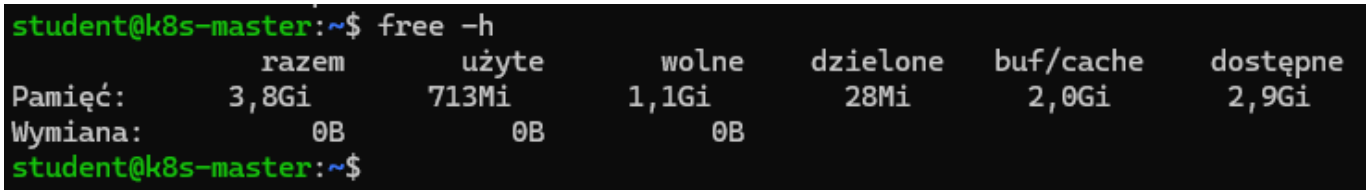
1. Etap 1: Utworzenie klastra Kubernetes (Milestone 1)

1.1. Środowisko testowe

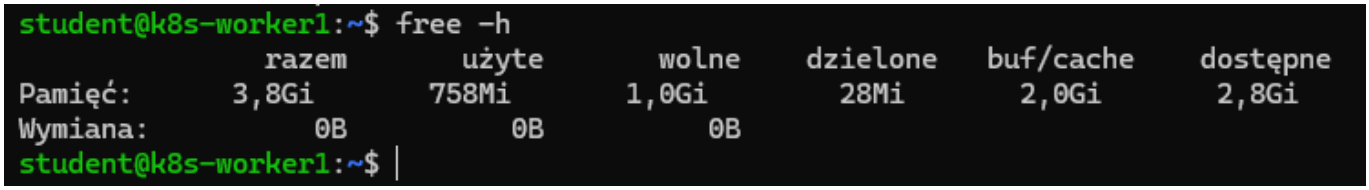
Klaster Kubernetes został uruchomiony na maszynach wirtualnych w środowisku VirtualBox. Na wszystkich węzłach zainstalowano system operacyjny Ubuntu 22.04 LTS (architektura amd64). Topologia klastra obejmuje jeden węzeł control-plane (`k8s-master`) oraz dwa węzły robocze (`k8s-worker1`, `k8s-worker2`).

1.2. Wyłączenie mechanizmu SWAP

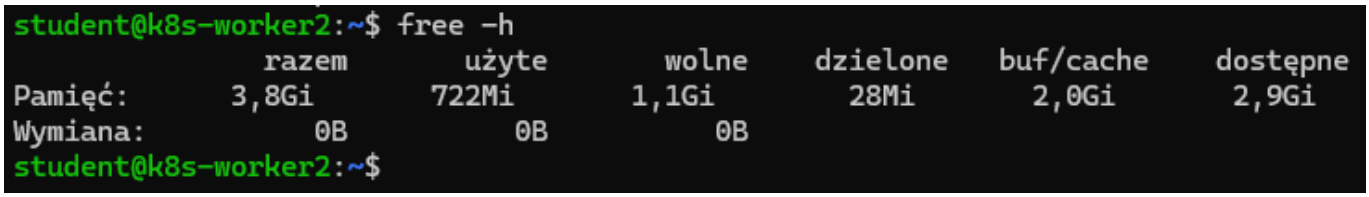
Kubernetes (kubelet) wymaga wyłączonego mechanizmu SWAP. Weryfikację stanu pamięci na wszystkich węzłach przedstawiono na rysunkach poniżej.



Rysunek 1. Weryfikacja wyłączonego SWAP na węźle master (`free -h`).



Rysunek 2. Weryfikacja wyłączonego SWAP na węźle worker1 (`free -h`).



Rysunek 3. Weryfikacja wyłączonego SWAP na węźle worker2 (`free -h`).

1.3. Konfiguracja parametrów jądra systemu

W celu zapewnienia poprawnego routingu pakietów IPv4 oraz filtrowania ruchu przechodzącego przez mosty sieciowe skonfigurowano parametry `sysctl` oraz załadowano moduł `br_netfilter`.

```
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.ipv4.ip_forward=1
net.bridge.bridge-nf-call-iptables=1
net.bridge.bridge-nf-call-ip6tables=1
EOF

sudo modprobe br_netfilter
sudo sysctl --system
```

1.4. Środowisko uruchomieniowe kontenerów

Zgodnie z wymaganiami projektu klaster wspiera Docker jako runtime kontenerów. Ponieważ Docker Engine nie implementuje natywnie interfejsu CRI, a wsparcie dockershim zostało usunięte z Kubernetes od wersji 1.24, zainstalowano dodatkowy komponent `cri-dockerd`, który zapewnia warstwę CRI dla Dockera [1].

1.5. Instalacja narzędzi Kubernetes

Na wszystkich węzłach zainstalowano narzędzia `kubeadm`, `kubelet` oraz `kubectl` zgodnie z oficjalną dokumentacją Kubernetes [2].

1.6. Inicjalizacja klastra na węźle control-plane

Klaster zainicjalizowano na węźle `k8s-master` poleceniem:

```
sudo kubeadm init \
  --apiserver-advertise-address=192.168.0.146 \
  --pod-network-cidr=10.244.1.0/16 \
  --service-cidr=10.96.1.0/12 \
  --cri-socket unix:///var/run/cri-dockerd.sock
```

Po inicjalizacji skonfigurowano dostęp `kubectl` zgodnie z komunikatem `kubeadm`.

```
student@k8s-master:~$ mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
  sudo chown $(id -u):$(id -g) $HOME/.kube/config
student@k8s-master:~$ kubectl get nodes
NAME          STATUS     ROLES           AGE   VERSION
k8s-master    NotReady   control-plane   8m    v1.34.2
```

Rysunek 4. Konfiguracja `kubectl` po `kubeadm init`.

1.7. Dołączenie węzłów roboczych

Węzły robocze zostały dołączone do klastra przy użyciu polecenia `kubeadm join` (z parametrem `--cri-socket` wskazującym `cri-dockerd`).

```
student@k8s-worker2:~$ sudo kubeadm join 192.168.0.146:6443 --token 5h3eys.8cfwbdtntck08now \
        --discovery-token-ca-cert-hash sha256:027a24001e7346ecf1b52ec59bd5eb1df6df8500103747bc91daa53203dbfeec \
        --cri-socket unix:///var/run/cri-dockerd.sock
[sudo] hasło użytkownika student:
[preflight] Running pre-flight checks
[preflight] Reading configuration from the "kubeadm-config" ConfigMap in namespace "kube-system"...
[preflight] Use 'kubeadm init phase upload-config kubeadm --config your-config-file' to re-upload it.
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/instance-config.yaml"
[patches] Applied patch of type "application/strategic-merge-patch+json" to target "kubeletconfiguration"
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 1.003807315s
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap

This node has joined the cluster:
* Certificate signing request was sent to apiserver and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.

student@k8s-worker2:~$
```

Rysunek 5. Dołączanie węzła roboczego do klastra Kubernetes.

1.8. Instalacja CNI – Cilium

W klastrze zainstalowano CNI Cilium, które było wykorzystywane w kolejnych etapach projektu do egzekwowania polityk sieciowych [3, 4]. Cilium zainstalowano z parametrem `--set ipam.mode=kubernetes`, dzięki czemu adresy IP podów będą przydzielane zgodnie z pulą *podCIDR* skonfigurowaną podczas inicjalizacji klastra poleceniem `kubeadm init`.

```
student@k8s-master:~$ kubectl get pods -n kube-system
NAME                                       READY   STATUS    RESTARTS        AGE
cilium-9wkfh                               1/1     Running   1 (4d20h ago)   4d20h
cilium-c7ttc                               1/1     Running   1 (4d20h ago)   4d20h
cilium-envoy-2hbzg                         1/1     Running   1 (4d20h ago)   4d20h
cilium-envoy-2ltv7                         1/1     Running   1 (4d20h ago)   4d20h
cilium-envoy-x9t8d                         1/1     Running   1 (4d20h ago)   4d20h
cilium-ggrkj                               1/1     Running   1               4d20h
cilium-operator-68bd8cc456-d88ss           1/1     Running   1 (4d20h ago)   4d20h
coredns-66bc5c9577-6dbmr                   1/1     Running   1 (4d20h ago)   4d20h
coredns-66bc5c9577-zxc4k                   1/1     Running   0               40m
etcd-k8s-master                            1/1     Running   1 (4d20h ago)   4d20h
kube-apiserver-k8s-master                  1/1     Running   1 (4d20h ago)   4d20h
kube-controller-manager-k8s-master         1/1     Running   1 (4d20h ago)   4d20h
kube-proxy-dsqpg                           1/1     Running   1 (4d20h ago)   4d20h
kube-proxy-jvfkk                           1/1     Running   1 (4d20h ago)   4d20h
kube-proxy-x94nf                           1/1     Running   1 (4d20h ago)   4d20h
kube-scheduler-k8s-master                  1/1     Running   1 (4d20h ago)   4d20h
student@k8s-master:~$
```

Rysunek 6. Weryfikacja działania Cilium w przestrzeni `kube-system`.

1.9. Weryfikacja działania klastra

Poprawne działanie klastra potwierdzono poprzez:

— sprawdzenie stanu węzłów (`kubectl get nodes -o wide`),
— weryfikację działania control-plane (`kubectl cluster-info`),
— potwierdzenie działania CNI Cilium.

```
student@k8s-master:~$ kubectl get nodes -o wide
NAME          STATUS   ROLES           AGE   VERSION   INTERNAL-IP     EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION     CONTAINER-RUNTIME
k8s-master    Ready    control-plane   25m   v1.34.2   192.168.0.146   <none>        Ubuntu 22.04.5 LTS   6.8.0-87-generic   docker://29.1.2
k8s-worker1   Ready    <none>          13m   v1.34.2   192.168.0.116   <none>        Ubuntu 22.04.5 LTS   6.8.0-87-generic   docker://29.1.2
k8s-worker2   Ready    <none>          12m   v1.34.2   192.168.0.185   <none>        Ubuntu 22.04.5 LTS   6.8.0-87-generic   docker://29.1.2
student@k8s-master:~$
```

Rysunek 7. Węzły klastra Kubernetes w stanie `Ready`.

```
student@k8s-master:~$ kubectl cluster-info
Kubernetes control plane is running at https://192.168.0.146:6443
CoreDNS is running at https://192.168.0.146:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
```

Rysunek 8. Potwierdzenie działającego klastra `kubectl cluster-info`

2. Etap 2: Wdrożenie aplikacji na klastrze Kubernetes

W ramach drugiego etapu projektu wdrożono architekturę trójwarstwową składającą się z frontendu, backendu oraz bazy danych. Każda z aplikacji została zdefiniowana jako obiekt typu *Deployment* z wymaganą liczebnością dwóch replik.

2.1. Konfiguracja Deploymentów

Do wdrożenia przygotowano plik `wdrozenie.yaml`, zawierający definicje dla trzech serwisów. Poniżej przedstawiono szczegóły konfiguracji poszczególnych komponentów:

— **Frontend**: Oparty na obrazie `node:20-alpine`. Aplikacja nasłuchuje na porcie 3000 i serwuje prostą odpowiedź HTTP. Kod serwera został wstrzyknięty bezpośrednio w definicję poda poprzez argumenty polecenia.
— **Backend**: Wykorzystuje obraz `nginx`. Zgodnie z wymaganiami projektowymi dla zespołu P1, serwer został skonfigurowany do nasłuchiwania na porcie **81**. Zmiana domyślnego portu 80 na 81 została osiągnięta poprzez edycję pliku konfiguracyjnego `default.conf` w locie, przy użyciu narzędzia `sed` wewnątrz kontenera.
— **Database**: Wykorzystuje obraz `redis` i działa na standardowym porcie 6379.

Poniżej przedstawiono plik konfiguracyjny odpowiedzialny za wdrożenie aplikacji:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: database
  labels:
    app: database
spec:
  replicas: 2
  selector:
    matchLabels:
      app: database
  template:
    metadata:
      labels:
        app: database
    spec:
      containers:
      - name: redis
        image: redis
        ports:
        - containerPort: 6379
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  labels:
    app: backend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
      - name: nginx
        image: nginx
        ports:
        - containerPort: 81
        command: ["/bin/sh", "-c"]
        args:
```

```
          - |
            sed -i 's/listen  *80;/listen 81;/' /etc/nginx/conf.d/default.conf && nginx -g 'daemon off;'
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  labels:
    app: frontend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
      - name: nodejs
        image: node:20-alpine
        ports:
        - containerPort: 3000
        command: ["/bin/sh", "-c"]
        args:
          - |
            echo 'var http = require("http");
            http.createServer(function (req, res) {
              res.write("Frontend dziala!");
              res.end();
            }).listen(3000);' > server.js && node server.js
```

2.2. Weryfikacja wdrożenia

Po zastosowaniu konfiguracji poleceniem `kubectl apply -f wdrozenie.yaml`, zweryfikowano poprawność utworzenia obiektów Deployment oraz status podów.

```
student@k8s-master:~/Pulpit$ kubectl apply -f wdrozenie.yaml
deployment.apps/database created
deployment.apps/backend created
deployment.apps/frontend created
```

Rysunek 9. Zastosowanie pliku wdrożeniowego i utworzenie obiektów Deployment

Wszystkie pody uzyskały status *Running*. Potwierdzono, że pody otrzymały adresy IP z puli zgodnej z przyjętą konfiguracją sieciową klastra (widoczne na rysunku 10), co świadczy o poprawnym działaniu sieci w klastrze.

```
student@k8s-master:~/Pulpit$ kubectl get pods -o wide
NAME                        READY   STATUS    RESTARTS   AGE     IP            NODE          NOMINATED NODE   READINESS GATES
backend-65cb8f578-7rcjt     1/1     Running   0          2m32s   10.244.2.209  k8s-worker2   <none>           <none>
backend-65cb8f578-z6ztw     1/1     Running   0          2m32s   10.244.1.106  k8s-worker1   <none>           <none>
database-c9b75ccc7-llt7d    1/1     Running   0          2m32s   10.244.1.9    k8s-worker1   <none>           <none>
database-c9b75ccc7-ps5qm    1/1     Running   0          2m32s   10.244.2.84   k8s-worker2   <none>           <none>
frontend-5bbcc64c9b-c98jm   1/1     Running   0          2m32s   10.244.2.166  k8s-worker2   <none>           <none>
frontend-5bbcc64c9b-wb2lk   1/1     Running   0          2m32s   10.244.1.38   k8s-worker1   <none>           <none>
```

Rysunek 10. Lista uruchomionych podów z widocznymi adresami IP oraz przypisaniem do węzłów

Rysunek 11 prezentuje wynik polecenia `kubectl get svc`. Widoczny adres IP **10.96.0.1** potwierdza, że klaster poprawnie wykorzystuje zdefiniowaną podczas inicjalizacji pulę adresową dla serwisów.

```
student@k8s-master:~/Pulpit$ kubectl get svc
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP   82m
```

Rysunek 11. Weryfikacja puli adresowej dla serwisów (Service CIDR)

2.3. Wstępne testy komunikacji (brak polityk)

Przed wdrożeniem reguł bezpieczeństwa przeprowadzono testy połączeń między podami, aby potwierdzić domyślne zachowanie sieci w klastrze Kubernetes. W standardowej konfiguracji (bez obiektów *NetworkPolicy*) ruch sieciowy jest nieograniczony.

Do testowania wykorzystano narzędzie `netcat` (polecenie `nc -zv`).

1. **Frontend → Backend**: Połączenie na porcie 81 powiodło się (ruch pożądany).
2. **Backend → Baza Danych**: Połączenie na porcie 6379 powiodło się.
3. **Frontend → Baza Danych**: Połączenie na porcie 6379 powiodło się (ruch niepożądany – frontend nie powinien mieć bezpośredniego dostępu do bazy).
4. **Baza Danych → Backend**: Połączenie na porcie 81 powiodło się (co stanowi lukę bezpieczeństwa).

```
student@k8s-master:~/Pulpit$ kubectl exec -it frontend-5bbcc64c9b-c98jm -- /bin/sh
/ # nc -zv 10.244.2.209 81
10.244.2.209 (10.244.2.209:81) open
/ # nc -zv 10.244.1.106 81
10.244.1.106 (10.244.1.106:81) open
```

Rysunek 12. Test połączenia: Frontend łączy się poprawnie z Backendem (port 81)

```
student@k8s-master:~/Pulpit$ kubectl exec -it backend-65cb8f578-7rcjt -- /bin/bash
root@backend-65cb8f578-7rcjt:/# nc -zv 10.244.1.9 6379
10.244.1.9: inverse host lookup failed: Unknown host
(UNKNOWN) [10.244.1.9] 6379 (?) open
root@backend-65cb8f578-7rcjt:/# nc -zv 10.244.2.84 6379
10.244.2.84: inverse host lookup failed: Unknown host
(UNKNOWN) [10.244.2.84] 6379 (?) open
root@backend-65cb8f578-7rcjt:/#
```

Rysunek 13. Test połączenia: Backend łączy się poprawnie z Bazą Danych (port 6379)

```
student@k8s-master:~/Pulpit$ kubectl exec -it frontend-5bbcc64c9b-c98jm -- /bin/sh
/ # nc -zv 10.244.1.9 6379
10.244.1.9 (10.244.1.9:6379) open
/ # nc -zv 10.244.2.84 6379
10.244.2.84 (10.244.2.84:6379) open
```

Rysunek 14. Zagrożenie bezpieczeństwa: Frontend ma bezpośredni dostęp do Bazy Danych

```
student@k8s-master:~/Pulpit$ kubectl exec -it database-c9b75ccc7-ps5qm -- /bin/sh
# nc -zv 10.244.2.209 81
10.244.2.209: inverse host lookup failed: Unknown host
(UNKNOWN) [10.244.2.209] 81 (?) open
# nc -zv 10.244.1.106 81
10.244.1.106: inverse host lookup failed: Host name lookup failure
(UNKNOWN) [10.244.1.106] 81 (?) open
#
```

Rysunek 15. Zagrożenie bezpieczeństwa: Baza Danych może inicjować połączenia do Backendu

3. Etap 3. Stworzenie polityk sieciowych (Network Policies):

Trzeci etap projektu obejmował zabezpieczenie komunikacji wewnątrz klastra poprzez implementację polityk sieciowych.

3.1. Założenia i implementacja

Zdefiniowano następujące reguły w pliku `polityki.yaml`:

1. **Backend (backend-policy)**:
   — **Ingress**: Dozwolony ruch przychodzący na port 81 wyłącznie z podów oznaczonych etykietą `app: frontend`.
2. **Frontend (frontend-policy)**:
   — **Ingress**: Ruch przychodzący dozwolony (nieograniczony, aby umożliwić dostęp użytkowników do aplikacji).
3. **Baza danych (db-policy)**:
   — **Ingress**: Dozwolony ruch przychodzący na port 6379 wyłącznie z podów oznaczonych etykietą `app: backend`.
   — **Egress**: Zablokowano cały ruch wychodzący, z wyjątkiem zapytań DNS (port 53 UDP) do serwera `kube-dns`, co jest niezbędne do poprawnego rozwiązywania nazw wewnątrz klastra.

Poniżej przedstawiono plik konfiguracyjny odpowiedzialny za wdrożenie polityki sieciowej:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-policy
spec:
  podSelector:
    matchLabels:
      app: database
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: backend
    ports:
    - protocol: TCP
      port: 6379
  egress:
  - to:
    - namespaceSelector: {}
      podSelector:
        matchLabels:
          k8s-app: kube-dns
    ports:
    - protocol: UDP
      port: 53
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-policy
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
    ports:
```

```
    - protocol: TCP
      port: 81
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: frontend-policy
spec:
  podSelector:
    matchLabels:
      app: frontend
  policyTypes:
  - Ingress
  ingress:
  - {}
```

3.2. Wdrożenie polityk sieciowych

Polityki zostały wdrożone w klastrze, a ich obecność zweryfikowano poleceniem `kubectl get networkpolicies`.

```
student@k8s-master:~/Pulpit$ kubectl apply -f polityki.yaml
networkpolicy.networking.k8s.io/db-policy created
networkpolicy.networking.k8s.io/backend-policy created
networkpolicy.networking.k8s.io/frontend-policy created
```

Rysunek 16. Wdrożenie zdefiniowanych polityk sieciowych

```
student@k8s-master:~/Pulpit$ kubectl get networkpolicies
NAME              POD-SELECTOR    AGE
backend-policy    app=backend     45s
db-policy         app=database    45s
frontend-policy   app=frontend    45s
```

Rysunek 17. Weryfikacja aktywnych polityk sieciowych w klastrze

Po wdrożeniu plików YAML z definicjami *NetworkPolicy*, ponownie przeprowadzono serię testów łączności, aby zweryfikować, czy zastosowane reguły działają zgodnie z założeniami projektowymi.

3.3. Testowanie polityk sieciowych:

3.3.1. Połączenia dozwolone

Zgodnie z oczekiwaniami, ruch niezbędny do działania aplikacji został przepuszczony:

```
student@k8s-master:~/Pulpit$ kubectl exec -it frontend-5bbcc64c9b-c98jm -- /bin/sh
/ # nc -zv 10.244.2.209 81
10.244.2.209 (10.244.2.209:81) open
/ # nc -zv 10.244.1.106 81
10.244.1.106 (10.244.1.106:81) open
/ #
```

Rysunek 18. Sukces: Polityka zezwala na ruch z Frontendu do Backendu

```
student@k8s-master:~/Pulpit$ kubectl exec -it backend-65cb8f578-7rcjt -- /bin/bash
root@backend-65cb8f578-7rcjt:/# nc -zv 10.244.1.9 6379
10.244.1.9: inverse host lookup failed: Host name lookup failure
(UNKNOWN) [10.244.1.9] 6379 (?) open
root@backend-65cb8f578-7rcjt:/# nc -zv 10.244.2.84 6379
10.244.2.84: inverse host lookup failed: Host name lookup failure
(UNKNOWN) [10.244.2.84] 6379 (?) open
root@backend-65cb8f578-7rcjt:/#
```

Rysunek 19. Sukces: Polityka zezwala na ruch z Backendu do Bazy Danych

3.3.2. Połączenia zablokowane

Próby nawiązania połączeń niezdefiniowanych w politykach zakończyły się niepowodzeniem (timeout), co potwierdza szczelność izolacji sieciowej:

```
student@k8s-master:~/Pulpit$ kubectl exec -it frontend-5bbcc64c9b-c98jm -- /bin/sh
/ # nc -zv -w 5 10.244.1.9 6379
nc: 10.244.1.9 (10.244.1.9:6379): Operation timed out
/ # nc -zv -w 5 10.244.2.84 6379
nc: 10.244.2.84 (10.244.2.84:6379): Operation timed out
/ #
```

Rysunek 20. Blokada: Próba połączenia Frontendu bezpośrednio do Bazy Danych zakończona Timeoutem

```
student@k8s-master:~/Pulpit$ kubectl exec -it database-c9b75ccc7-ps5qm -- /bin/sh
# nc -zv 10.244.2.209 81
10.244.2.209: inverse host lookup failed: Host name lookup failure
^C
# nc -zv -w 5 10.244.2.209 81
10.244.2.209: inverse host lookup failed: Host name lookup failure
(UNKNOWN) [10.244.2.209] 81 (?) : Connection timed out
# nc -zv -w 5 10.244.1.106 81
10.244.1.106: inverse host lookup failed: Host name lookup failure
(UNKNOWN) [10.244.1.106] 81 (?) : Connection timed out
#
```

Rysunek 21. Blokada: Baza Danych nie może inicjować połączeń do Backendu

Wynik „Operation timed out” widoczny na powyższym zrzucie (uzyskany przy użyciu flagi `-w 5` dla netcata) świadczy o tym, że pakiety są skutecznie odrzucane przez mechanizm filtrowania sieci CNI, a nie po prostu odrzucane przez aplikację (co skutkowałoby błędem „Connection refused”).

3.4. Refleksja nad politykami bezpieczeństwa

Wprowadzenie polityk sieciowych w środowisku Kubernetes jest kluczowe. Domyślna konfiguracja sieci w Kubernetes (płaska sieć) pozwala na swobodną komunikację każdego poda z każdym, co stwarza ryzyko w przypadku ataku.

Zastosowane w projekcie ograniczenia przynoszą następujące korzyści:

— **Minimalizacja wektora ataku**: Kompromitacja frontendu nie daje atakującemu bezpośredniego dostępu do bazy danych, ponieważ ruch ten jest blokowany na poziomie sieciowym.

— **Izolacja komponentów**: Baza danych nie może inicjować połączeń na zewnątrz (np. w celu pobrania złośliwego oprogramowania lub eksfiltracji danych), co zostało osiągnięte poprzez restrykcyjną politykę *Egress*.

— **Zgodność z regulacjami i normami (Compliance)**: Wymuszanie ścisłych reguł przepływu danych jest często wymagane przez standardy bezpieczeństwa (np. RODO), zapewniając, że dane wrażliwe są dostępne tylko dla autoryzowanych serwisów (w tym przypadku backendu).

Ochrona danych w środowisku kontenerowym nie może polegać wyłącznie na zabezpieczeniach brzegowych (firewall). Network Policies przenoszą bezpieczeństwo bezpośrednio do poziomu aplikacji, co jest niezbędne w dynamicznym środowisku chmurowym.

Literatura

[1] Kubernetes Documentation, *Container runtimes*, https://kubernetes.io/docs/setup/production-environment/container-runtimes/

[2] Kubernetes Documentation, *Installing kubeadm*, https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/install-kubeadm/

[3] Kubernetes Documentation, *Cilium Network Policy*, https://kubernetes.io/docs/tasks/administer-cluster/network-policy-provider/cilium-network-policy/

[4] Cilium Documentation, *Installing Cilium*, https://docs.cilium.io/en/stable/gettingstarted/k8s-install-default/